Introduction to word embeddings

Matthieu Labeau - Associate Professor, Telecom Paris, IPP

APM_4AI12_TP: Introduction to Text Processing - 04th April 2025

Representing words

Vector Semantics

How to represent **word meaning** ?

- Using a vector based on the **distribution of context** !
  → Based on the **distributional** hypothesis: *contextual information is sufficient to represent linguistic objects*

Wittgenstein (Philosophical Investigations, 1953)

For a large class of cases [...] the meaning of a word is its use in the language.

Firth (1957)

You shall know a word by the company it keeps

- Meaning are embedded into a space: these vectors are called **Word Embeddings**: usual representation of meaning in NLP
- In our case, word meaning is represented by a vector describing *the distribution of other words in the neighborhood*

A little introduction to lexical semantics

In linguistics, study of **word meaning**:

- The meaning of a word (often simplified to its lemma) can consist in **several senses** (when it's the case, the lemma is polysemous - an important task in NLP is word sense disambiguation)
- There exists many relationships between word meanings: for example, two words that share a sense are **synonyms**
- Words can be *similar* but also only *related* (do you think of examples ?)
- How to evaluate word similarity ?
  - Ask humans ? Many datasets of similarities, like *SimLex-999*

Evaluating Semantic Models with (Genuine) Similarity Estimation, (Hill et al, 2015)

- clothes/closet: 1.96
- coast/shore: 9.00

  - Look at the annotator guidelines !

Words can be vectors

- Reverse our previous matrix: words are *vectors of documents*
- Or change the **context**: words can ben *vectors of sentences*, or *phrases*
  - Why not only use **surrounding words** ?
- The simplest possible context for words ? Other words ! Gives the *co-occurence* matrix:

	I	the	down	walked	street	avenue	walk	ran	city
I	0	6	5	2	2	2	2	1	1
the	6	2	6	2	2	3	3	1	1
down	5	6	0	2	2	2	2	1	1
walked	2	2	2	0	1	1	0	0	1
street	2	2	2	1	0	0	0	1	0
avenue	2	3	2	1	0	0	1	0	0
walk	2	3	2	0	0	1	0	0	1
ran	1	1	1	0	1	0	0	0	0
city	1	1	1	1	0	0	1	0	0

→ The **cosine distance** can be used to compute word similarity

Finding word relationships

Let's assume that $\mathbf{w}_1$ and $\mathbf{w}_2$ have high cosine similarity:

- That implies they have the same distributional context !
- What can we know about their relationship: *synonym, antonym* ?

Now, let's assume that $c(w_1, w_2)$ is high. What could it imply ?

- Examples:

	I	¬ I
the	123	1245
¬ the	987	9437

	Computer	¬ Computer
Semantics	7	19
¬ Semantics	31	11735

- What values should we look at to understand how *significant* co-occurence counts are ?

Pointwise Mutual Information

Pointwise Mutual Information (PMI): Evaluate how **unexpected** is two words co-occuring !

- PMI: log-ratio of the *joint probability* of two words and the *product of their marginals*:

$$\text{PMI}(w_i, w_j) = \log\left(\frac{P(w_1, w_2)}{P(w_1)P(w_2)}\right)$$

- This probability corresponds to the observed frequency:

$$\text{PMI}(\mathbf{M}, w_1, w_2) = \log\left(\frac{M_{w_1,w_2} \times \left(\sum_{k=1}^{n}\sum_{l=1}^{n} M_{k,l}\right)}{\left(\sum_{l=1}^{n} M_{w_1,l}\right) \times \left(\sum_{k=1}^{n} M_{k,w_2}\right)}\right)$$

- Values range in $[-\infty, \infty]$ but negative values are unreliable (why ?)
  - We clip them to 0 (***Positive*** *pointwise mutual information*)
- Extreme values are still an issue (division by small values) !
- As TF-IDF, can be used to compute similarities.

Difficulties with count-based embeddings

- Being of the **size of the vocabulary** $\mathcal{V}$, vectors can get huge: memory and computation issues
- Vectors are **sparse** - many empty dimensions
- When computing similarity, should all dimensions (represented by other words) matter the same ?
- We only have access to direct relationships between words (*first-order co-occurence*)
- Skewed frequency is always a challenge

Three types of solutions - which can be combined:

- Reduce the vocabulary $\rightarrow$ *lemmatization*, *removing stop words*...
- Next idea: **re-weight** the vectors $\rightarrow$ *PPMI*
- And then: obtain **dense representations**

Better representations: dimension reduction

- Cosine similarity on symbolic representations does not work well: should all dimensions (represented by words) matter the same ?
- How can we represent documents by doing **more than counting words** ?

Idea: take advantage of the **latent structure** in the **association between** the set of **words** and the set of **documents**

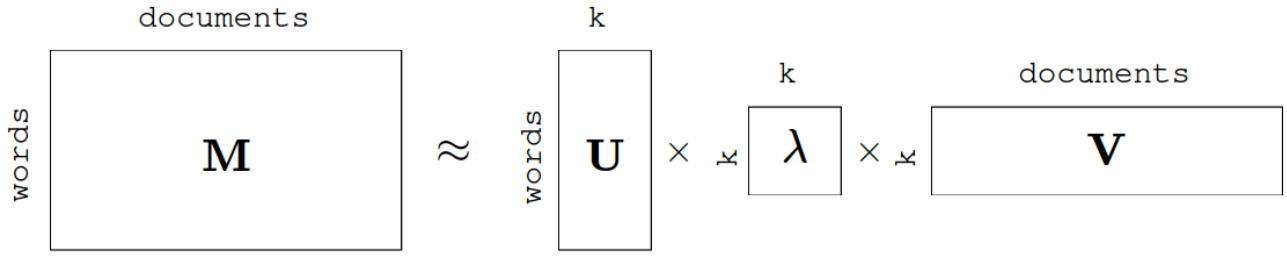
- First method: linearly reduce the dimension to put forward higher-order relationships
  $\rightarrow$ Use Singular Value Decomposition (SVD)

$$\mathbf{M} = \mathbf{U}\lambda\mathbf{V}^{\mathsf{T}}$$

  - λ diagonal matrix, with eigenvalues - ordered
  - $\mathbf{U}, \mathbf{V}$ orthogonal; eigenvectors
  - Keep the k first columns of $\mathbf{U}$, for the k largest eigenvalues, to obtain embeddings $\in \mathbb{R}^k$
  - But very costly (quadratic in memory, cube in flops)

Latent Semantic Analysis

This method is called **Latent Semantic Analysis**:



- The new space is interpreted as **topics**: this is the first method for *topic modeling*
- Reminder: SVD rotates the axis along directions of largest variations among the documents (generalized least-squares method)
- Useful for information retrieval, but also if we need **higher-order features than word counts**
- Can help to represent documents in topic space for classification !
  - Besides the cost, it must be re-run if you add new documents.

Naïve Bayes as a probabilistic graphical model

The **Naïve Bayes generative model** assumes that a set of documents $\mathcal{D} = [d_i]_{i=1}^N$, belonging to classes $c_i \in \mathcal{C}$ are generated through the following:

1. **Sample a class label** c from the prior distribution on $\mathcal{C}$:
   - $c \sim \text{Categorical}(\mu)$
2. **Generate words** $w_1, w_2, ..., w_l$ independently given c:
   - For token $w_k, k = 1, ..., l$, draw $w_k \sim \text{Categorical}(\phi_{\mathbf{c}})$

The **joint probability** of a document $d = (w_1, w_2, ..., w_l)$ and class c is:

$$\mathbb{P}(d, c) = \mu_c \prod_{k=1}^{l} \phi_c^{w_k} = \mathbb{P}(c) \prod_{k=1}^{l} \mathbb{P}(w_k \mid c)$$

Where:

- $\mathbb{P}(c) = \mu_c$ is the **prior probability** of class c.
- $\mathbb{P}(w_k \mid c) = \phi_c^{w_k}$ is the **class-conditional probability** of word w_k.

Goal: finding the best parameters μ and $\phi_{\mathbf{c}}, \forall c \in \mathcal{C}$ with respect to $\mathcal{D}$

Classification with Naïve Bayes

Multinomial naive Bayes classifier:

- Applying **Baye's rule** and the **naïve independance assumption**, we get a (*prior* $\times$ *likelihood*) decomposition:

$$\hat{c} = \text{argmax}_{c \in \mathcal{C}} \left[\mathbb{P}(c) \prod_{k=1}^{l} \mathbb{P}(w_k|c) \right]$$

- Idea: we compute **empirically** $\mathbb{P}(w|c)$ as the frequency of w among all documents $d \in \mathcal{D}$ of class c:

$$\phi_c^w = \mathbb{P}(w|c) = \frac{count_{\mathcal{D}}(w, c)}{\sum_{w' \in \mathcal{V}} count_{\mathcal{D}}(w', c)}$$

$$\text{and } \mu_c = \mathbb{P}(c) = \frac{count_{\mathcal{D}}(c)}{N}$$

Writing the likelihood of the parameters $\mathcal{L}(\mu, \phi)$ and adding constraints, we could obtain the same result through **Maximum Likelihood Estimation** (*how* ?)

Classification with Naïve Bayes

Practical points: Use **log-probabilities** (why ?) and Laplace smoothing

Training Algorithm: Given data $\mathcal{D}$ and classes $\mathcal{C}$

- Create the vocabulary $\mathcal{V}$ from documents $\mathcal{D}$
- From $\mathcal{D}$: For each class $c \in \mathcal{C}$: compute the log-prior
  $\log \mathbb{P}(c) = \log \frac{count_{\mathcal{D}}(c)}{N}$
  - For each word $w \in \mathcal{V}$ compute the log-likelihood
    $\log \mathbb{P}(w|c) = \frac{count_{\mathcal{D}}(w,c)}{\sum_{w' \in \mathcal{V}} count_{\mathcal{D}}(w',c)}$

Inference Algorithm: Given document d to classify:

- For each class $c \in \mathcal{C}$: $S(c) = \log \mathbb{P}(c)$
  - For each position $k \in d$:
    If $w_k \in \mathcal{V}$: $S(c) = S(c) + \log \mathbb{P}(w_k|c)$
- Return $\text{argmax}_{c \in \mathcal{C}} S(c)$

Probabilistic Latent Semantic Analysis

Uses generative modelling to do a *better, interpretable* LSA
$\rightarrow$ Documents are represented over a latent topic distribution $[z_k]_{k=1}^K$

1. **Pick a document** d_i with probability $\mathbb{P}(d_i) = \frac{1}{N}$
2. **Sample a topic** z_k from $\mathbb{P}(z_k|d_i) = \beta_i^k$
3. **Generate a words** w_j given z_k, from $\mathbb{P}(w_j|z_k) = \phi_k^j$

The **joint probability** of a document d_i and a word w_j is then:

$$\mathbb{P}(d_i, w_j) = \frac{1}{N} \sum_{k=1}^{K} \phi_k^j \beta_i^k = \mathbb{P}(Dd_i) \sum_{k=1}^{K} \mathbb{P}(w_j \mid z_k) \mathbb{P}(z_k \mid d_i)$$

Goal: finding the best parameters β and ϕ by maximizing the likelihood

$$\mathcal{L}(\beta, \phi) = \prod_{i=1}^{N} \prod_{j=1}^{V} \mathbb{P}(w_j|d_i)$$

(Solved with the **Expectation-Maximization** algorithm: next wednesday)

Language modelling

Back to NLP Applications

Let's look at a few NLP applications:

- Speech Recognition: we need to know that $\mathbb{P}(\text{'recognize speech'}) > \mathbb{P}(\text{'wreck a nice beach'})$!

Back to NLP Applications

Let's look at a few NLP applications:

- Speech Recognition: we need to know that $\mathbb{P}$('recognize speech') $>$ $\mathbb{P}$('wreck a nice beach') !
- Machine Translation: $\mathbb{P}$('I like avocados') $>$ $\mathbb{P}$('I like lawyers')

Back to NLP Applications

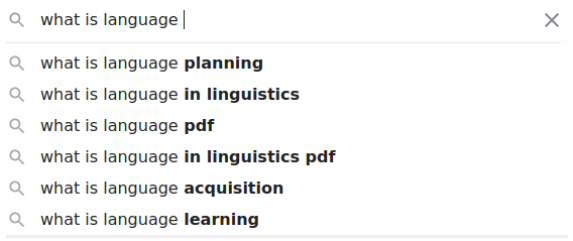
Let's look at a few NLP applications:

- Speech Recognition: we need to know that
  $\mathbb{P}$('recognize speech') $>$ $\mathbb{P}$('wreck a nice beach') !
- Machine Translation: $\mathbb{P}$('I like avocados') $>$ $\mathbb{P}$('I like lawyers')
- Grammar/Spelling Correction:
  $\mathbb{P}$('He likes avocados') $>$ $\mathbb{P}$('He like avocados')

Back to NLP Applications

Let's look at a few NLP applications:

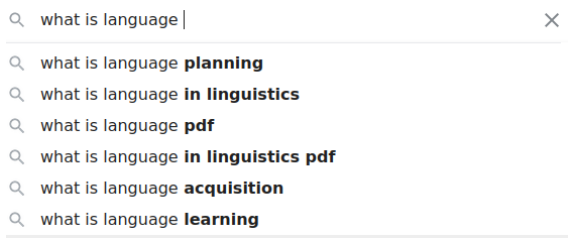
- Speech Recognition: we need to know that
  $\mathbb{P}$('recognize speech') $>$ $\mathbb{P}$('wreck a nice beach') !
- Machine Translation: $\mathbb{P}$('I like avocados') $>$ $\mathbb{P}$('I like lawyers')
- Grammar/Spelling Correction:
  $\mathbb{P}$('He likes avocados') $>$ $\mathbb{P}$('He like avocados')
- And directly for ranking:



Back to NLP Applications

Let's look at a few NLP applications:

- Speech Recognition: we need to know that
  $\mathbb{P}$('recognize speech') $>$ $\mathbb{P}$('wreck a nice beach') !
- Machine Translation: $\mathbb{P}$('I like avocados') $>$ $\mathbb{P}$('I like lawyers')
- Grammar/Spelling Correction:
  $\mathbb{P}$('He likes avocados') $>$ $\mathbb{P}$('He like avocados')



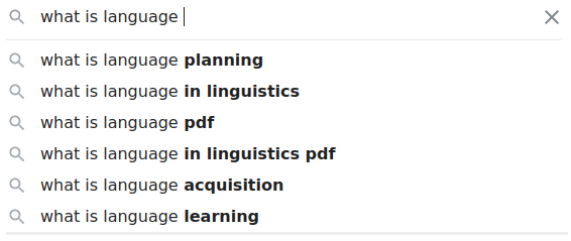
- And directly for ranking:

- Goal: **assign the larger probability to the best option !**

Back to NLP Applications

Let's look at a few NLP applications:

- Speech Recognition: we need to know that
  $\mathbb{P}$('recognize speech') $>$ $\mathbb{P}$('wreck a nice beach') !
- Machine Translation: $\mathbb{P}$('I like avocados') $>$ $\mathbb{P}$('I like lawyers')
- Grammar/Spelling Correction:
  $\mathbb{P}$('He likes avocados') $>$ $\mathbb{P}$('He like avocados')
- And directly for ranking:



- Goal: **assign the larger probability to the best option !**
- It is also necessary for Optical Character Recognition, Information Retrieval, Summarization, Question Answering... and many other tasks (including anything that necessitate **Text Generation**)

Probabilities on language: Language Modeling

- A **Language Model** (**LM**) estimates the probabilities of text sequences. **How ?**

$$\mathbb{P}(\text{What is language modeling ?}) = \ ?$$

Usually, we *count* ! Let's assume that we divide our sentence in **words**.

- **Chain rule**: decompose in smaller parts:

$$\mathbb{P}(\text{What is}) = \mathbb{P}(\text{What})\mathbb{P}(\text{is} \mid \text{What})$$

$$\mathbb{P}(\text{What is language}) = \mathbb{P}(\text{What is})\mathbb{P}(\text{language} \mid \text{What, is})$$

$$\mathbb{P}(\text{What is language}) = \mathbb{P}(\text{What})\mathbb{P}(\text{is} \mid \text{What})\mathbb{P}(\text{language} \mid \text{What, is})$$

- **Formally**,

$$\mathbb{P}(w_1, ..., w_m) = \prod_{i=1}^{m} \mathbb{P}(w_i | w_{<i})$$

Statistics on words: n-grams

- A **n-gram** is a sequence of n successive items. For *words*, we can see them as an ordered sequence looking $n-1$ words into the past.

- Just using word frequencies: **1-grams** (*unigrams*)
- If you know the previous word: **2-grams** (*bigrams*)
- If you know the two previous words: **3-grams** (*trigrams*)
  $\rightarrow$ of course, this depends on the *tokenization* !

- How would these apply to previous examples ?

- An interesting tool: Google Ngram Viewer

Simplifying assumptions

- We use the simplifying **Markov** assumption: the probability depends upon a fixed number of words (the *order*).

$$\text{(Order 1)} \quad \mathbb{P}(\text{modeling}|\text{What is language}) \approx \mathbb{P}(\text{modeling}|\text{language})$$

$$\text{(Order 2)} \quad \mathbb{P}(\text{modeling}|\text{What is language}) \approx \mathbb{P}(\text{modeling}|\text{is, language})$$

$$\vdots$$

$$\text{(Order k)} \quad \mathbb{P}(w_i|w_1, ..., w_{i-1}) \approx \mathbb{P}(w_i|w_{i-k+1}, ..., w_{i-1})$$

- With the chain rule, we obtain simple language models:

$$\text{(Order 0: Unigram)} \quad \mathbb{P}(w_1, ..., w_m) \approx \prod_{i=1}^{m} \mathbb{P}(w_i)$$

$$\text{(Order 1 : Bigram)} \quad \mathbb{P}(w_1, ..., w_m) \approx \mathbb{P}(w_1) \prod_{i=2}^{m} \mathbb{P}(w_i|w_{i-1})$$

n-gram Language Models

Usual models

- The most used is arguably the trigram:

$$\mathbb{P}(w_1, ..., w_m) \approx \mathbb{P}(w_1)P(w_2|w_1)\prod_{i=3}^{m}\mathbb{P}(w_i|w_{i-2}, w_{i-1})$$

- Then, we obtain the following:
  $\mathbb{P}(\text{What is language modeling}) \approx \mathbb{P}(\text{What}) \times \mathbb{P}(\text{is} \mid \text{What}) \times$
  $\mathbb{P}(\text{language} \mid \text{What, is}) \times \mathbb{P}(\text{modeling} \mid \text{is, language})$

- Models are often extended to 4-grams, 5-grams. More is not really sustainable (*Why ?*)
  - Due to long-term dependancies, this is in general insufficient to model language !
  - However, n-gram models are still used in numerous applications

Model definition

- A n-gram model is **parametric**, with the number of parameters $\sim O(|\mathcal{V}|^n)$

- The model is defined by *every possible conditional probability that can be computed given its vocabulary* $\mathcal{V}$. Hence, a n-gram model is defined by the values of:

$$\theta = \{\mathbb{P}(w_n|w_1, ..., w_{n-1}), \forall(w_1, w_2, ..., w_n) \in \mathcal{V}^n\}$$

- How to compute the probability of a word w_n given a context of previous words $h = (w_1, w_2, ..., w_{n-1})$?

$\rightarrow \forall(w_1, w_2, ..., w_n) \in \mathcal{V}^n$

$$\mathbb{P}(w_n|w_{1:n-1}) = \frac{C(w_1, ..., w_{n-1}, w_n)}{C(w_1, ..., w_{n-1})} = \frac{C(w_1, ..., w_{n-1}, w_n)}{\sum_{w' \in \mathcal{V}} C(w_1, ..., w_{n-1}, w')}$$

Indeed, the sum of all n-gram counts that start with a given sequence $w_{1:n-1}$ is equal to the $(n-1)$-gram count for that sequence !

Bigram model: an example

- Let us assume the following corpus: what are the **bigram probabilities** ?

I walked down the street
I walk down the the avenue
I walked down the avenue
I ran down the street
I walk down the city

- Let's count ! First, what is the vocabulary ?

Bigram model: an example

- Let us assume the following corpus: what are the **bigram probabilities** ?

I walked down the street
I walk down the the avenue
I walked down the avenue
I ran down the street
I walk down the city

- Let's count ! First, what is the vocabulary ?

$$\rightarrow \mathcal{V} = \{\text{I}, \text{the}, \text{down}, \text{walked}, \text{street}, \text{avenue}, \text{walk}, \text{ran}, \text{city}\}$$

Counting bigrams

- We can compute the count matrix $\mathbf{C}$, where $\mathbf{C}_{i,j} = C((w_i, w_j))$:

	I	the	down	walked	street	avenue	walk	ran	city
I	0	0	0	2	0	0	2	1	0
the	0	1	0	0	2	2	0	0	1
down	0	5	0	0	0	0	0	0	0
walked	0	0	2	0	0	0	0	0	0
street	0	0	0	0	0	0	0	0	0
avenue	0	0	0	0	0	0	0	0	0
walk	0	0	2	0	0	0	0	0	0
ran	0	0	1	0	0	0	0	0	0
city	0	0	0	0	0	0	0	0	0

Estimating parameters

- Applying the previous formula, we compute the matrix $\mathbf{P}$, where $\mathbf{P}_{i,j} = \mathbb{P}_\theta(w_j|w_i)$:

	I	the	down	walked	street	avenue	walk	ran	city
I	0	0	0	2 / 5	0	0	2/5	1/5	0
the	0	1/3	0	0	1/3	1/3	0	0	1/6
down	0	1	0	0	0	0	0	0	0
walked	0	0	1	0	0	0	0	0	0
street	0	0	0	0	0	0	0	0	0
avenue	0	0	0	0	0	0	0	0	0
walk	0	0	1	0	0	0	0	0	0
ran	0	0	1	0	0	0	0	0	0
city	0	0	0	0	0	0	0	0	0

- What can we notice ? What are some potential issues ?
- What probabilities does this model give to the sentences of the corpus ?

Model evaluation

- **Extrinsic** evaluation:
  - Apply the models that we want to compare on a task, and use the related metric ! (e.g, *speech recognition* and *word error rate*)
    $\rightarrow$ Time-consuming, task-dependant.

- **Intrinsic** evaluation: **Perplexity**
  - A measure of uncertainty: how surprised is the model by the data ?

$$\text{Perplexity}(w_1, ..., w_m) = \sqrt[m]{\prod_{i=1}^{m} \frac{1}{\mathbb{P}_\theta(w_i | w_{i-n+1}, ..., w_{i-1})}}$$

  - Can be interpreted as a branching factor
  - But is tied to a particular vocabulary $\mathcal{V}$

Estimating parameters: theoretical consideration

- Estimating parameters = getting **counts** from a corpus, normalizing them into **probabilities** = Computing the *relative frequency* !

- It is by definition the **Maximum Likelihood Estimation** (**MLE**).
  - This maximizes the likelihood of the corpus as a whole given the constraints of the model
  - Of course, our bigram model is bad... but it gives the corpus *the best likelihood possible* for any a bigram model

- Perplexity is a simple function of the cross-entropy:

$$\text{Perplexity}(w_1, ..., w_m) = 2^{-\frac{1}{m}\sum_{i=1}^{m} \log(\mathbb{P}_\theta(w_i|w_{i-n+1},...,w_{i-1}))}$$

- The model maximizes the likelihood of the data used to estimate its parameter (training data), hence it minimizes its perplexity
  - We want it to give low perplexity to new (testing) data

Perplexity: an example

- What is the perplexity of the **bigram model** we previously defined on its **training data** ?
- Assume an **unigram model** estimated computing relative frequencies with the same vocabulary, on the same data: what is its perplexity ?
- Now, assume that the model is unigram, but attributes **uniform probabilities** to each word: what is its perplexity ?

- How do you interpret those results ? Can we compare those perplexities ?

- What happens if we try to compute the probabilities / perplexity of the following sentences ?

I walked down the city

I ran down the avenue

I run down the street

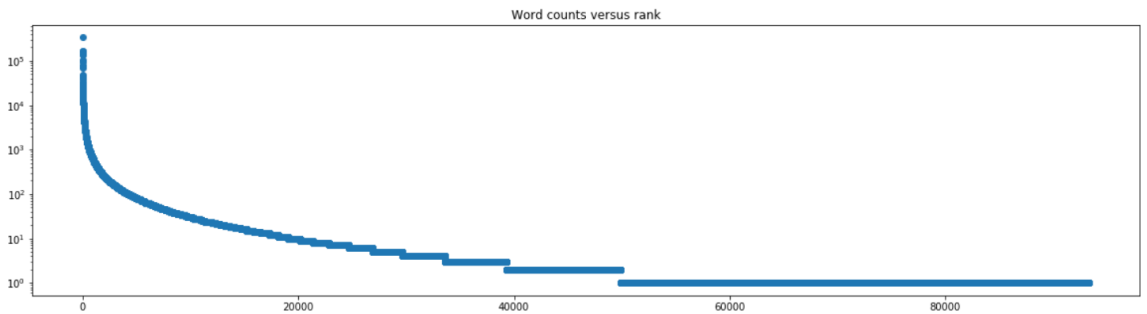
Generalization

- Huge issue: **Zero probability n-grams**
  - In a corpus, occuring n-grams represent a very small part of the possible n-grams.

In Shakespeare's work (Example from Dan Jurafsky)

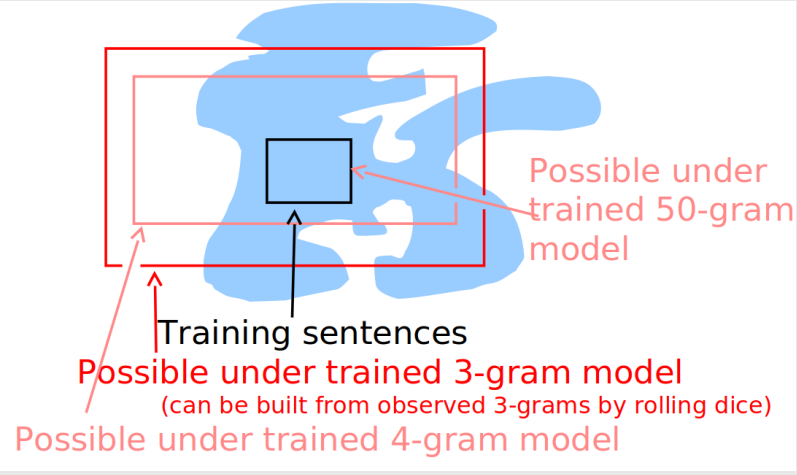
- 884,647 tokens and a vocabulary of 29,066 words
- 300K out of the 844M bigrams appear $(0,04\%)$!

- Tied to **Zipf's Law**: $frequency \propto 1/rank$



Parenthesis : Acceptability

From Jason Eisner's NLP class



- Combining acceptable n-grams can easily give unacceptable English.
- Is there a way to rule them out ?
  - Is increasing the amoung of training text enough ?
- Are n-grams enough ?

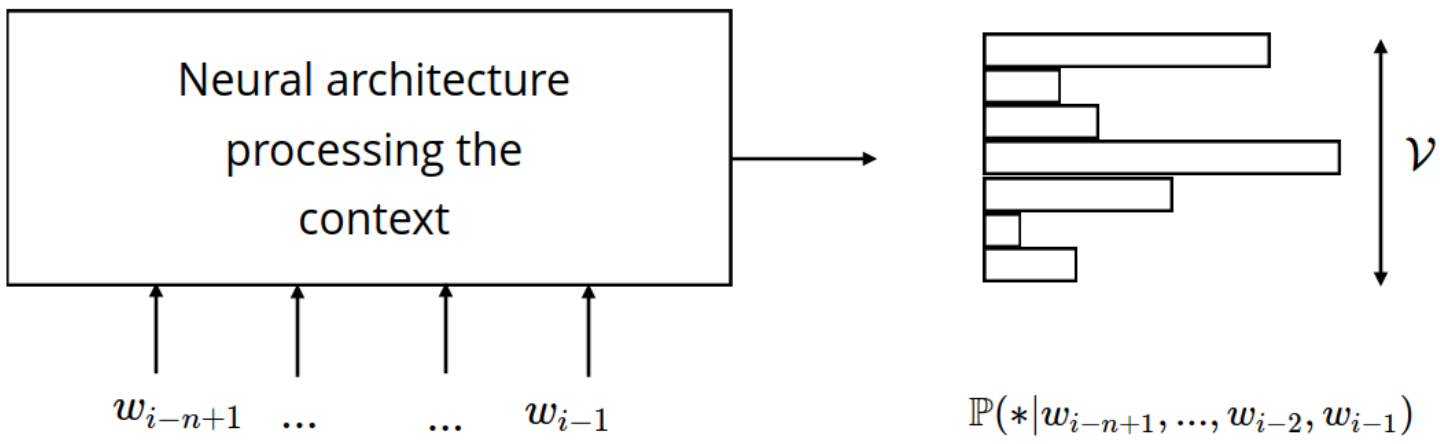
- Direction not taken in this class: *grammar modeling*
- We'll stick to statistics, within the current paradigm (solution: scaling *way way* up)

Neural Probabilistic Language Models

n-gram *neural* models

- Instead of computing $\mathbb{P}_\theta(w_i|w_{i-n+1}, ..., w_{i-1})$ with corpus statistics, we can teach a neural network to **predict** these probabilities

- This is divided in two parts:
  - Processing the context words - how it is done depends on the neural architecture used:



  - Obtaining an output probability distribution for the next word - it's the same whatever the model, we are classifying over $\mathcal{V}$

NPLM: A first model

A Neural Probabilistic Language Model (Bengio et al, 2003)

A similar model was first applied to speech recognition (Schwenk and Gauvain, 2002) and machine translation.

Main ideas:

- **Continuous word vectors**: Each input and output word is represented by a vector of dimension $d \ll |\mathcal{V}|$ taking values in $\mathbb{R}$, rather than being discrete
- **Continuous probability function**: The probability of the next word is expressed as a continuous function of the features of the word in the current context - using a neural network
- **Joint learning**: The parameters of the word representations, and the probability function are learnt jointly.

NPLM: Joint learning

Why should it work ?

- **Continuity**: the probability function is smooth, implying that a small change in the context or word vector will induce a small change in word probability.
- **Distributional hypothesis**: words appearing in similar contexts should have similar representations.

- Hence, the updates caused by having the following sentence:

  A dog was running in a room

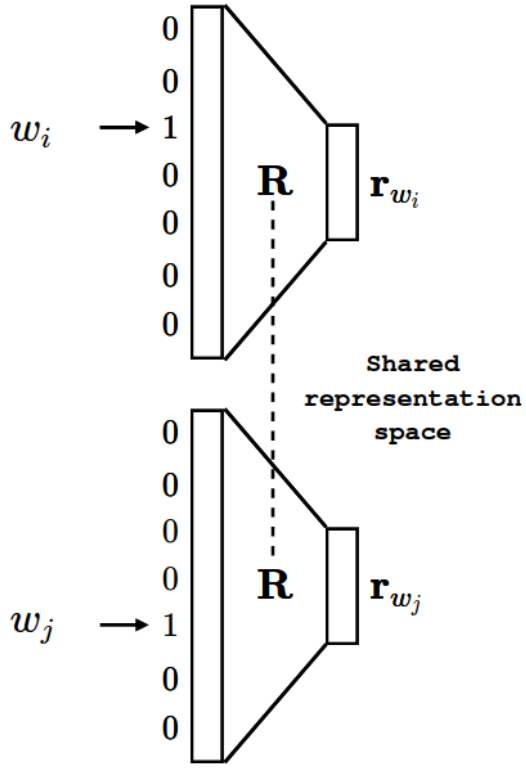
  in the training data will increase the probability of all 'neighbor' sentences. Having also the following sentence:

  The cat was running in a room

  will make the features of the words (dog, cat) get close to each other.

Neural model: projecting words

- Create a layer that is the vocabulary $\mathcal{V}$: the input is a **one-hot vector**.
- This layer is densely connected to a smaller continuous layer, of dimension d_w
- The parameters of the weight matrix $\mathbf{R}$ are what we call the **word embeddings**.
- Now, we assume working with a 3-gram (w_{i-2}, w_{i-1}, w_i): we need to project the two first words to predict the third.

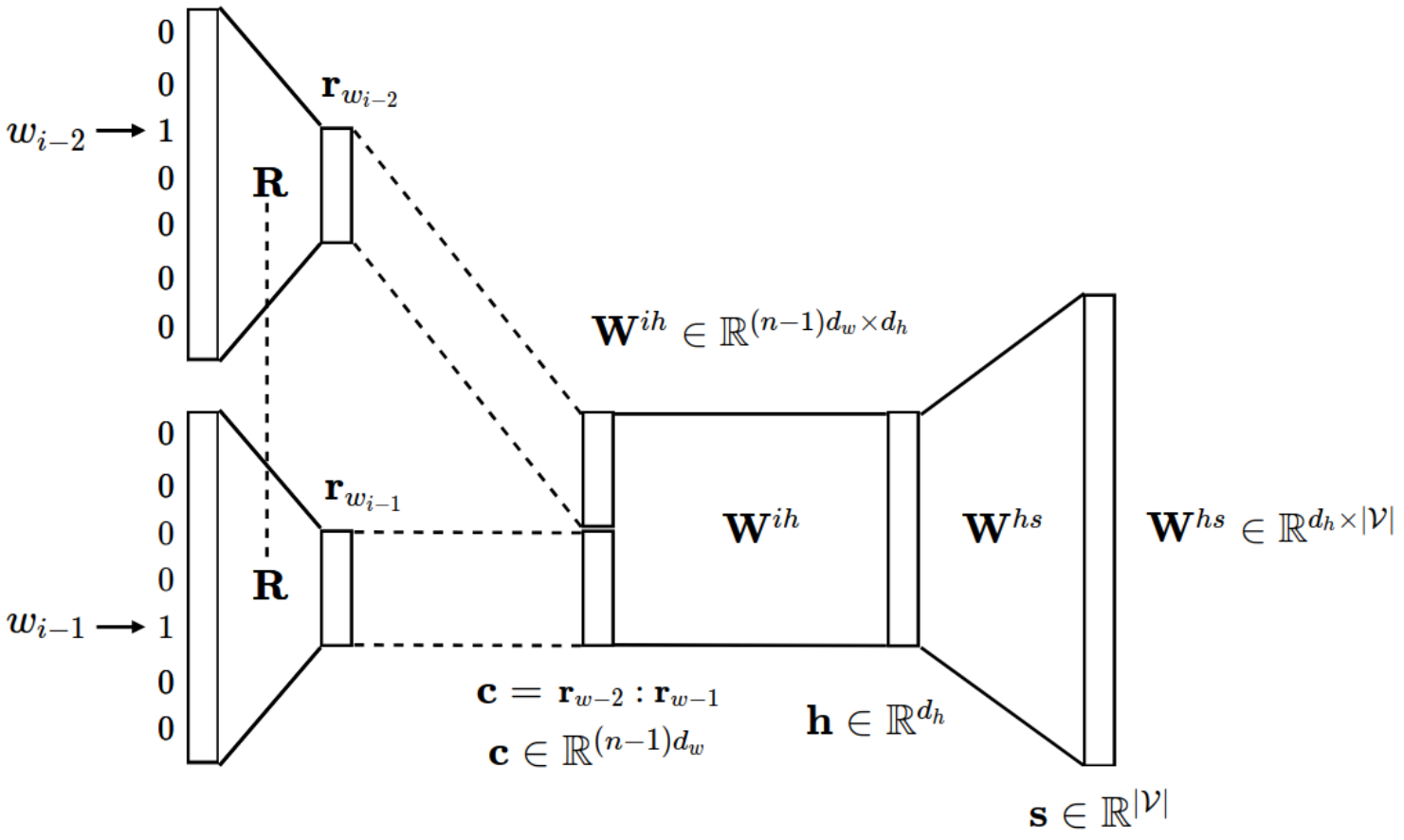


Neural model: Obtaining scores

- Given the context representation $\mathbf{c}$, create a hidden representation:

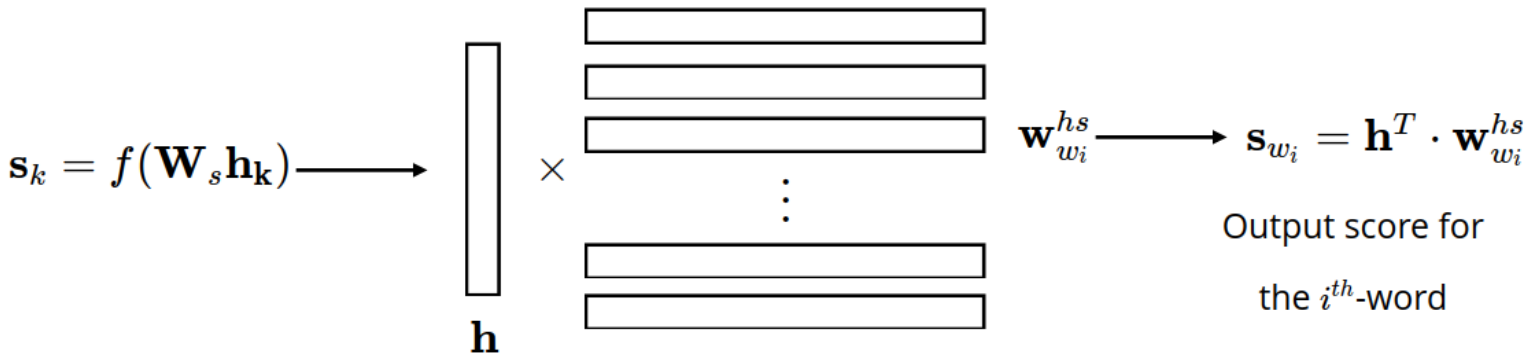
$$\mathbf{h} = \phi(\mathbf{W}^{ih}\mathbf{c})$$

- Then, obtain scores for all words in $\mathcal{V}$ given $\mathbf{h}$: $\mathbf{s} = \mathbf{W}^{hs}\mathbf{h}$



Neural model: Obtaining scores

The prediction layer can be seen as a dot product between $\mathbf{h}$ and output word embeddings $[\mathbf{w}_k^{hs}]_{k=1}^{|\mathcal{V}|}$:

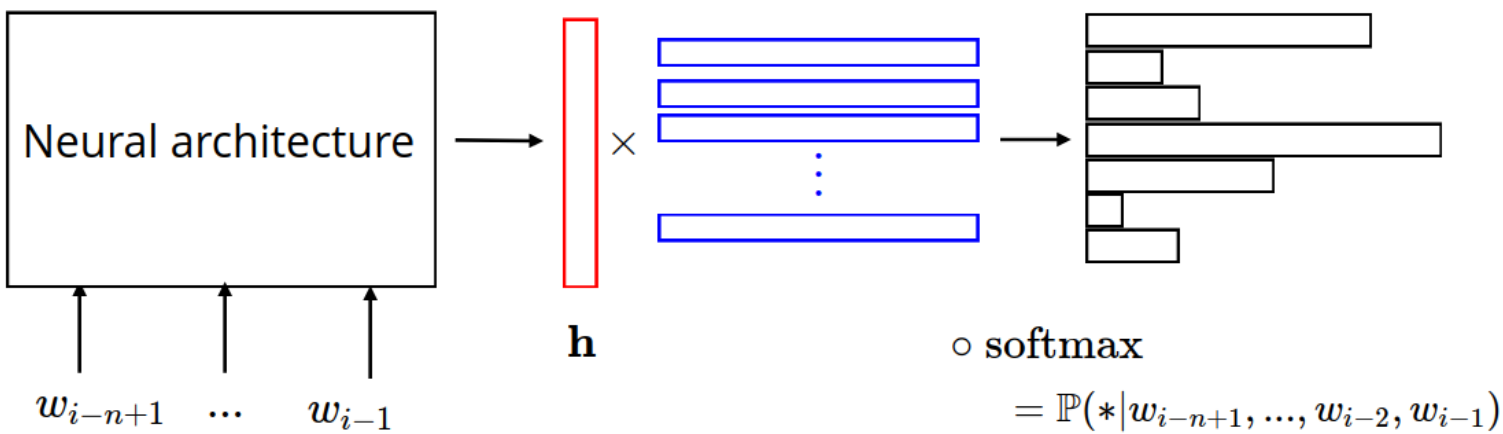


Neural model: Computing probabilities

- The goal of the neural architecture is here to get a vector representation $\mathbf{h}_i$ for the context input words $w_{i-n+1}, ..., w_{i-1}$.

- We use this representation against vector representations of all possible output o words $\mathbf{w}_o^{hs}$ in $\mathcal{V}$ to estimate their probabilities. We use the softmax function:

$$\mathbb{P}(o|w_{i-n+1}, ..., w_{i-1}) = \frac{\exp(\mathbf{h}_i^T \mathbf{w}_o^{hs})}{\sum_{l=1}^{|\mathcal{V}|} \exp(\mathbf{h}_i^T \mathbf{w}_l^{hs})}$$



Neural model: Training

- The input representations, hidden weights, and output representations are learned jointly: $\theta = (\mathbf{R}, \mathbf{W}^{ih}, \mathbf{W}^{hs})$

- We want the model output probability distribution $\mathbb{P}_\theta$, at timestep (i), to get close to the ground truth:

$$\mathbb{P}_\theta^{(i)}(*|w_{<i}) \rightarrow \text{ one-hot}(w_i) = \mathbb{P}_*^{(i)}$$

since we know that the next word is w_i.

- We look to minimize the distance between the two distributions:

$$d\left(\begin{bmatrix}0\\ \vdots \\ 1 \\ \vdots \\ 0\end{bmatrix}, \begin{bmatrix}\mathbb{P}_\theta^{(i)}(1|w_{<i})\\ \vdots \\ \mathbb{P}_\theta^{(i)}(w_i|w_{<i}) \\ \vdots \\ \mathbb{P}_\theta^{(i)}(|\mathcal{V}||w_{<i})\end{bmatrix}\right) ? \longrightarrow \begin{aligned} &\textbf{Cross-entropy}(\mathbb{P}_*^{(i)}, \mathbb{P}_\theta^{(i)}(*|w_{<i})) \\ &= -\sum_{k=1}^{|\mathcal{V}|} \mathbb{P}_*^{(i)}(k) log(\mathbb{P}_\theta^{(i)}(k|w_{<i})) \\ &= -log(\mathbb{P}_\theta^{(i)}(w_i|w_{<i}))\ ! \end{aligned}$$

Neural model: Training

- By minimizing this cross-entropy, at each step, we minimize the **negative log-likelihood** of the data sample $(w_i, w_{<i})$
- On all data samples $\mathcal{D} = (w_i)_{i=1}^m$, this is the Maximum-Likelihood Estimation (MLE) objective:

$$NLL(\theta) = -\sum_{i=1}^{m} \log(\mathbb{P}_\theta^{(i)}(w_i|w_{<i}))$$

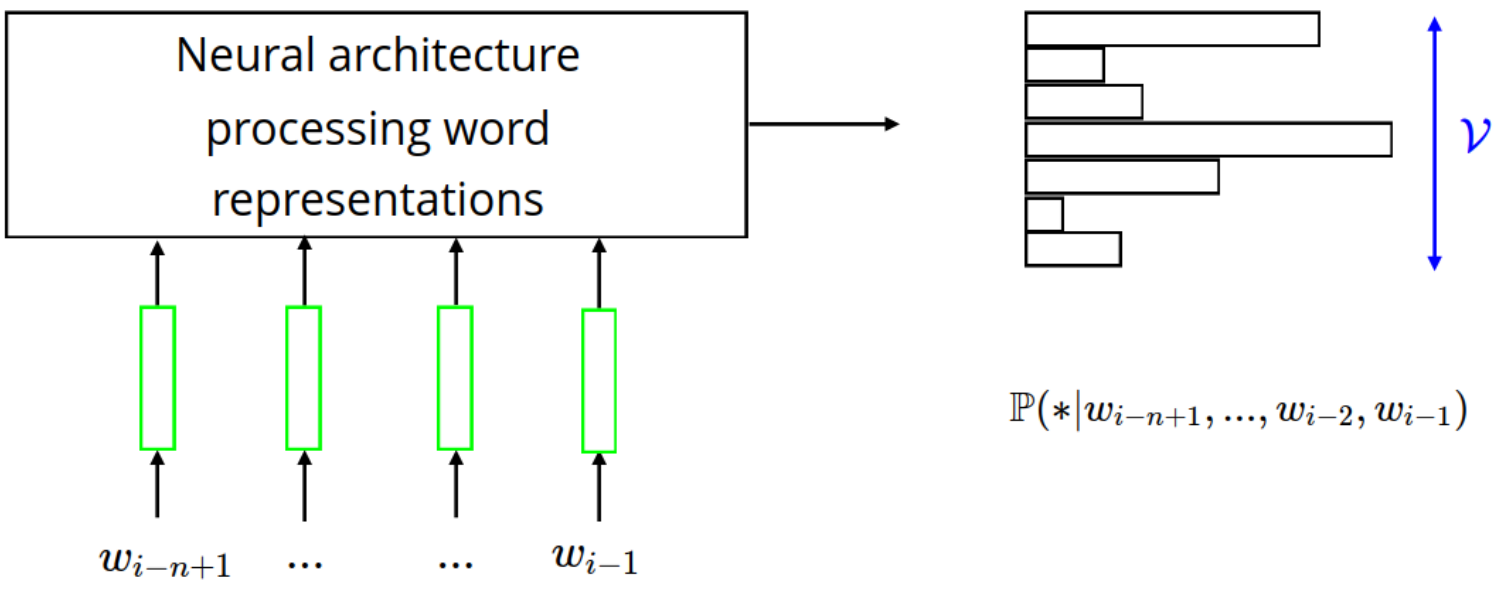
- Remark: minimizing the cross-entropy is equivalent to minimizing the Kullback-Leibler divergence
- We can note again that the perplexity is a simple function of the cross-entropy:

$$\text{Perplexity}(w_1, ..., w_m) = \sqrt[n]{\prod_{i=1}^{m} \frac{1}{\mathbb{P}_\theta^{(i)}(w_i|w_{<i})}} = 2^{-\frac{1}{m}\sum_{i=1}^{m} \log(\mathbb{P}_\theta^{(i)}(w_i|w_{<i}))}$$

$\rightarrow$ We are directly minimizing perplexity when training

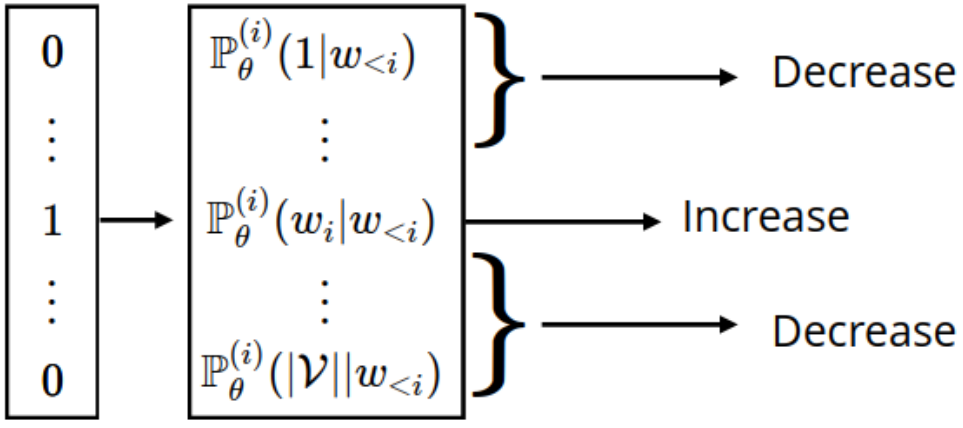
Neural model: Assessment

- Probability estimation is based on the similarity among the word vectors
- Projecting in continuous spaces reduces the **sparsity issue**
- Increasing the number of input words does not change much the complexity of the model
- The bottleneck is the output vocabulary size !



Neural model: Learning bottleneck

- During learning, probabilities are modified as follow:



The updates are computed using the following gradient:

$$\frac{\delta}{\delta\theta}\log(\mathbb{P}_\theta^{(i)}(w_i|w_{<i})) = \frac{\delta}{\delta\theta}\mathbf{s}_{w_i} - \sum_{w\in\mathcal{V}}\mathbb{P}_\theta^{(i)}(w|w_{<i})\frac{\delta}{\delta\theta}\mathbf{s}_w$$

- The first term increases the conditional log-likelihood of w_i given $w_{<i}$
- The second decreases the conditional log-likelihood of all the other words $w \in \mathcal{V}$ - and implies a double summation on $\mathcal{V}$

$\rightarrow$ The softmax causes this computational bottleneck !

Learning word embeddings

Learning Word Embeddings: why ?

Before Word2vec, semantic word representations exist in several forms:

From Frequency to Meaning: Vector Space Models of Semantics (Turney et al, 2010)

- From text-document matrices (*TF-IDF, LSA..*)
- From word-context matrices (*Co-occurences, PPMI..*)
- From more complex relationnal patterns, that can handle word order

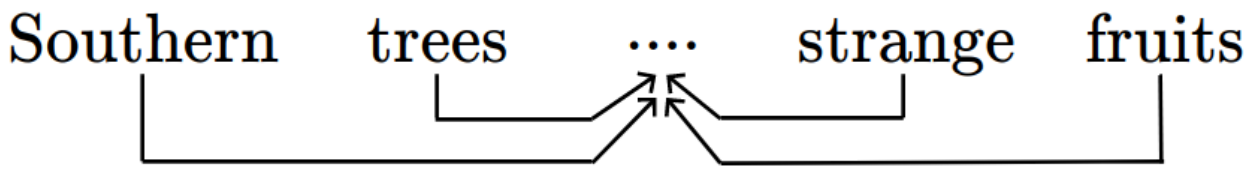
- All based on simply storing frequencies in a big tensor
  - All of these are **costly**

- Solution: learn to encode similarities in the vectors themselves
  - Initializing **dense word vectors** with random parameters
  - Train those parameters for words appearing together having similar vectors: **Word2vec**

Word2vec: Architecture

For the example *'Southern trees bear strange fruits'*:

- Use all context: $\mathbb{P}(w_t|w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2})$?



- Let's simplify it as much as possible. Assuming:
  - Context word representations $\mathbf{C} \in \mathbb{R}^{d \times |\mathcal{V}|}$
  - Output word representations $\mathbf{W} \in \mathbb{R}^{d \times |\mathcal{V}|}$

$$\mathbf{h} = \mathbf{C} \times \left(\begin{bmatrix}0\\0\\1\\0\\0\\0\\0\end{bmatrix} + \begin{bmatrix}1\\0\\0\\0\\0\\0\\0\end{bmatrix} + \begin{bmatrix}0\\0\\0\\0\\0\\1\\0\end{bmatrix} + \begin{bmatrix}0\\0\\0\\0\\0\\0\\1\end{bmatrix} \right) \quad (\in \mathbb{R}^d)$$

$$\mathbf{o} = softmax(\mathbf{h} \times \mathbf{W}) \quad (\in \mathbb{R}^{|\mathcal{V}|})$$

Word2vec: CBOW

This is the **Continuous bag-of-words** (CBOW) architecture

- Output:

$$\mathbb{P}(\text{bear}|w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}) = \mathbf{o}_{bear} = \frac{\exp(\mathbf{h}^T \mathbf{c}_{bear})}{\sum_{l=1}^{|\mathcal{V}|} \exp(\mathbf{h}^T \mathbf{c}_l)}$$

- Training:

$$\mathbb{P}(\text{bear}|w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}) = \mathbf{o}_{bear} \rightarrow \text{ one-hot(bear)} = \mathbb{P}_*^{(\text{bear})}$$

- Noting $\theta = \{\mathbf{W}, \mathbf{C}\}$ the parameters of the model:

$$d\left(\begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} \mathbb{P}_\theta(\text{word}_1) \\ \vdots \\ \mathbb{P}_\theta(\text{bear}) \\ \vdots \\ \mathbb{P}_\theta(\text{word}_{|\mathcal{V}|}) \end{bmatrix}\right) ? \longrightarrow \begin{aligned} &\textbf{Cross-entropy}(\mathbb{P}_*^{(\text{bear})}, \mathbb{P}_\theta) \\ &= -\sum_{k=1}^{|\mathcal{V}|} \mathbb{P}_*^{(\text{bear})}(k) \log(\mathbb{P}_\theta(k)) \\ &= -\log(\mathbb{P}_\theta(bear)) = -\log(\mathbf{o}_{\text{bear}}) \,! \end{aligned}$$

- Minimizing this cross-entropy $\Leftrightarrow$ minimize the negative log-likelihood of the data sample *'Southern trees bear strange fruits'*

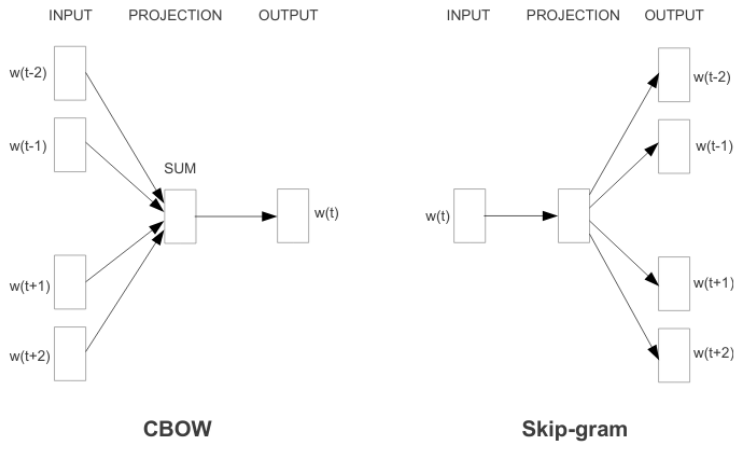
Word2vec: Skip-gram

- With a dataset $\mathcal{D} = (w_i)_{i=1}^N$ and a window of m words,

$$J_{MLE}^{CBOW}(\theta) = -\sum_{i=1}^{N} \log(\mathbb{P}_\theta(w_i | w_{i-m}, ..., w_{i-1}, w_{i+1}, ..., w_{i+m}))$$

- Other possible architecture, the **Skip-gram**:

$$J_{MLE}^{SG}(\theta) = -\sum_{i=1}^{N} \sum_{-m<j<m \atop j \neq 0} \log(\mathbb{P}_\theta(w_{i+j} | w_i))$$



(*What would be the interest of using this one ?*)

Word2vec: Too slow

Softmax gradient updates are slow and costly: with the skip-gram,

$$\frac{\delta}{\delta\theta}\log(\mathbb{P}_\theta(w_{i+j}|w_i)) = \frac{\delta}{\delta\theta}\mathbf{s}_{w_{i+j}} - \sum_{k=1}^{|\mathcal{V}|}\mathbb{P}_\theta(w_k|w_j)\frac{\delta}{\delta\theta}\mathbf{s}_{w_k}$$

- The first term increases the conditional log-likelihood of w_{i+j} given w_i
- The second decreases the conditional log-likelihood of all $w_k \in \mathcal{V}$

How to avoid computing any $\sum_{k=1}^{|\mathcal{V}|}$?

- Replace the task by **binary classification**: predicting the right word ?

$$\mathbb{P}_\theta(w_{i+j}|w_i) = \sigma(\mathbf{s}_{w_{i+j}})$$

- Only provides the **positive contribution** to the conditional log-likelihood:

$$\frac{\delta}{\delta\theta}\log(\mathbb{P}_\theta(w_{i+j}|w_i)) = (1-\sigma(\mathbf{s}_{w_{i+j}}))\frac{\delta}{\delta\theta}\mathbf{s}_{w_{i+j}}$$

- Let's add the negative contribution by **sampling** $k \ll |\mathcal{V}|$ **wrong words**

Word2vec: Accelerating training

Trick: how to **estimate an unknown distribution** $f(x)$ for which we have i.i.d samples (=*our data*) using a specified distribution $g(x)$ (=*word frequencies*) ?

- Assign class $Y = 1$ to samples from $f(x)$
- Assign class $Y = 0$ to samples from $g(x)$
- Assuming the same number of samples:

$$\mu(x) = \mathbb{E}(Y|x) = \frac{f(x)}{f(x) + g(x)} = \frac{\frac{f(x)}{g(x)}}{1 + \frac{f(x)}{g(x)}}$$

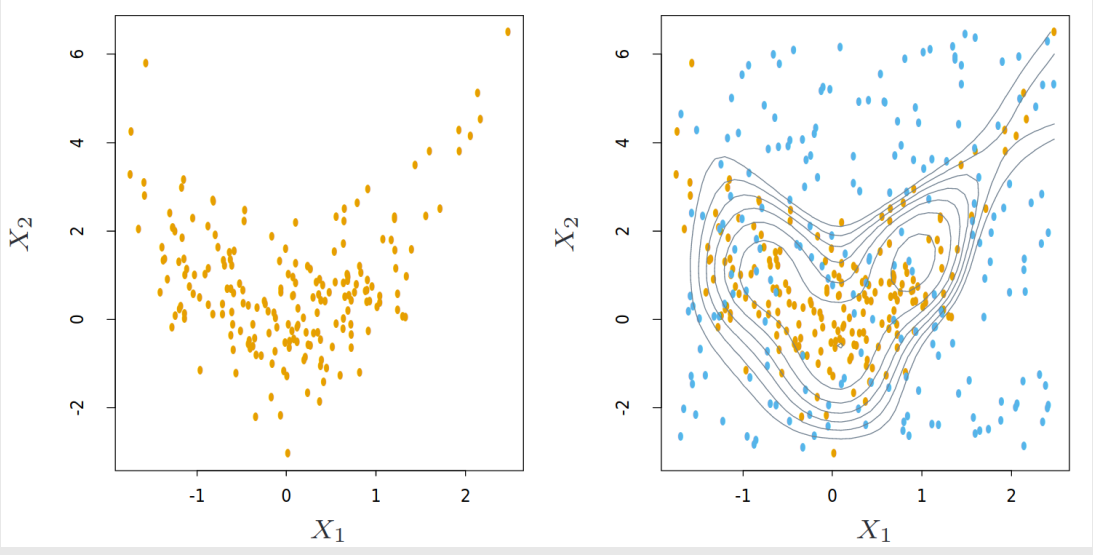
- We can reverse this and obtain an estimate $\hat{f}(x)$ from $\hat{\mu}(x)$:

$$\hat{f}(x) = g(x)\frac{\hat{\mu}(x)}{1 - \hat{\mu}(x)}$$

Word2vec: Density Ratio

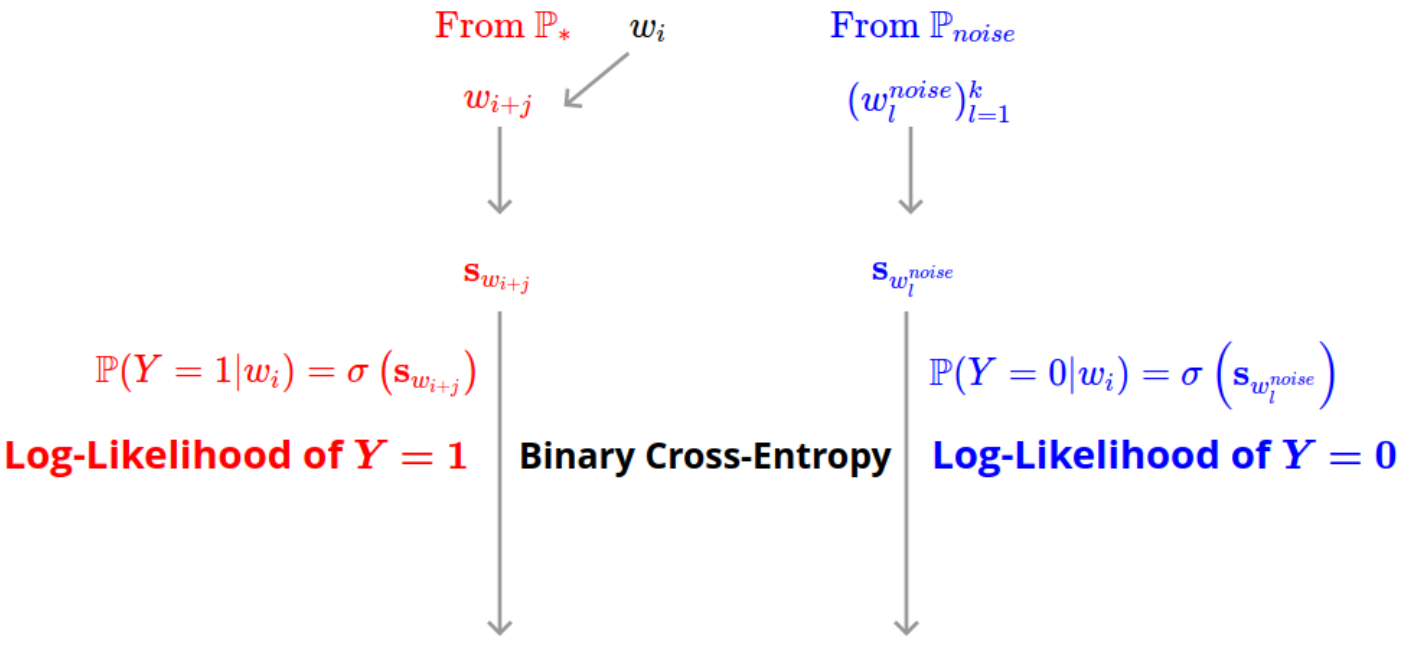
This is the **density-ratio trick**:

Elements of Statistical learning", 14.2.4 (Friedman et al, 2001)



- Directly estimate the **ratio of densities** $\frac{f(x)}{g(x)}$
- The accuracy of the estimation depends on $g(x)$
  $\rightarrow$ It should be able to reveal the properties of $f(x)$!

Word2vec: Negative sampling



$$J_{NS}^{SG}(\theta) = -\sum_{i=1}^{N}\sum_{j\neq 0}^{-m<j<m}\left(\log\sigma(\mathbf{s}_{w_{i+j}}) + \sum_{l=1}^{k}\log\sigma(\mathbf{s}_{w_l^{noise}})\right)$$

$$= -\sum_{i=1}^{N}\sum_{j\neq 0}^{-m<j<m}\left(\log\sigma(\langle\mathbf{w}_{w_{i+j}}, \mathbf{c}_{w_i}\rangle) + \sum_{l=1}^{k}\log\sigma(-\langle\mathbf{w}_{w_l^{noise}}, \mathbf{c}_{w_i}\rangle)\right)$$

- Very efficient, but requires some tricks: **subsampling of frequent words** in the noise distribution (*why ?*)

$$\mathbb{P}_{noise}(w) = (\text{freq}(w))^{\frac{3}{4}}$$

Word2Vec: summary

- Our training objective is to **predict a word from its context**
  - This is self-supervised learning
- We have a lot of data to exploit: we need to be able to scale up

$\rightarrow$ We adapt a simple **log-linear parametric model**

- We don't need complex modeling - we need efficiency
  - We just add words (*CBOW*) or consider them independantly (*Skip-gram*)
- We don't care about outputting a probability distribution
  - We use **negative sampling** to **approximate** the MLE objective
    - This is a form of *contrastive learning*

Count-based vs Prediction-based

- **Distributional** learning: *Words are similar if they appear in similar contexts*
- **Distributed** learning: Looking for *common attributes* to represent words with
    - Embed them in low dimension !

Word2vec (Skip-gram or CBOW):

- Choose the embedding space (number of parameters)
- Distributional learning is prediction based
- Scales efficiently with the quantity of data But, it does not efficiently use available information (like PMI)...

Count-based methods + SVD:

- We have seen that applying SVD to co-occurence counts gives disproportionate importance to large counts

$\rightarrow$ Can we get the best of both worlds ?

GloVe

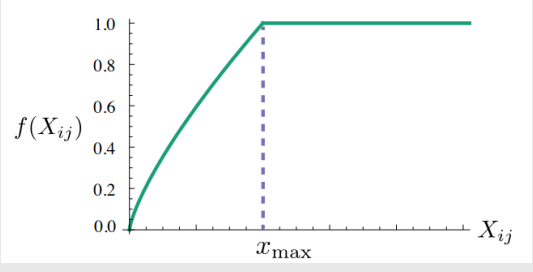
Idea: **directly learn word embeddings by predicting word PPMI values**

GloVe: Global Vectors for Word Representation (Pennington et al, 2014)

$$J_{Glove}(\theta) = \sum_{w_i, w_j \in \mathcal{V}} f\left(\mathbf{M}_{w_i,w_j}\right)\left(\mathbf{w}_{w_i}^{\top}\mathbf{w}_{w_j} - \log \mathbf{M}_{w_i,w_j}\right)^2$$

- $\theta = \{\mathbf{W}\}$
- f : scaling function - diminish the importance of frequent words

$$f(x)\begin{cases}(x/x_{\max})^{\alpha} & \text{if } x < x_{\max} \\ 1 & \text{otherwise}\end{cases}$$

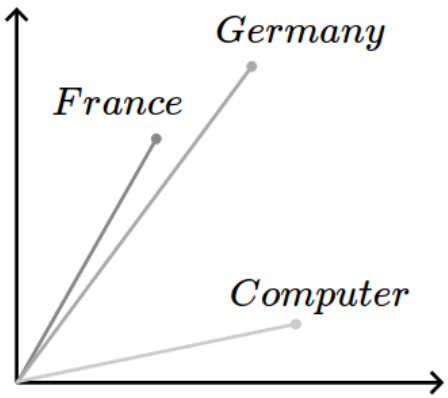


- Very fast training
- Scales to very large corpora

Evaluate word embeddings: similarity

- *Intrisic* or *extrinsic* evaluation
- **Intrinsic** evaluation:
  - Can help understand the method/data
  - Fast to compute
  - However, no notion of how useful the method is

→ Usually, by **word similarity**



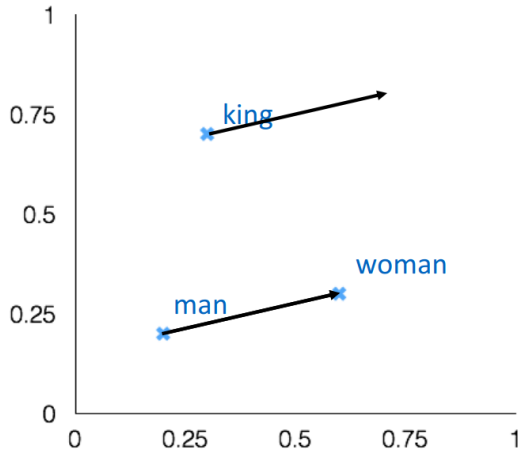
$$\text{cosine}(\mathbf{w}_i, \mathbf{w}_j) = \frac{\mathbf{w}_i \bullet \mathbf{w}_j}{||\mathbf{w}_i||\ ||\mathbf{w}_j||}$$

- Use datasets of pairs of words with human judgement (SimLex-999)
- Measure *Spearman Rank Correlation*

Properties: Analogy

- We can observe that **linear** word representations relation ships can capture meaning: for example,

$$\text{queen} = \underset{w_i \in \mathcal{V}}{\text{argmax}}[cos(\mathbf{w}_{w_i}, \mathbf{w}_{woman} - \mathbf{w}_{man} + \mathbf{w}_{king})]$$



- This also applies to other patterns in language. Let's check !
  $\rightarrow$ Look at the *visualization* notebook.

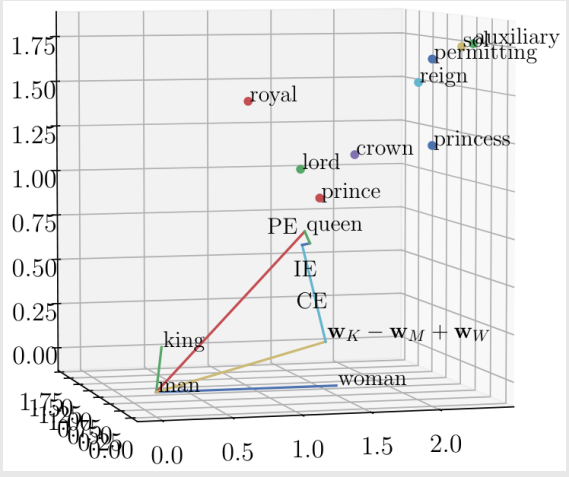
On explaining analogies

Many research works on the **linear relationships between word embeddings**:

Analogies Explained: Towards Understanding Word Embeddings (Allen et al, 2019)

What if the information is **not linear** ?

- See paraphrases as word transformations
- Write analogies as linear transformations
- + many (identified) error terms !



Towards Understanding Linear Word Analogies (Ethayarajh et al, 2019)

- Derive theoretical conditions for analogies to hold
  - **Co-occurrence Shifted PMI Theorem**
- Confirms previous observations (Scalar product of words $\approx$ PMI)

Which method works best ?

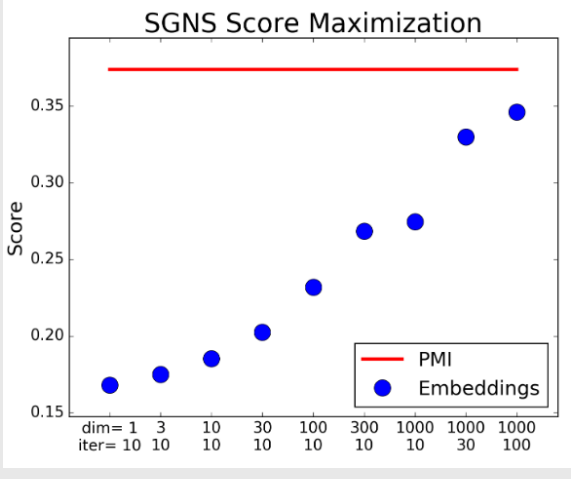
Main idea: all of these methods extract the same kind of information

Neural Word Embeddings as implicit matrix factorization (Melamud et al, 2017)

Finding: the PMI matrix maximizes word2vec's *skip-gram with negative sampling* objective function

Information-Theory Interpretation of the Skip-Gram Negative-Sampling Objective Function (Melamud et al, 2017)

Finding: Word2vec's *skip-gram with negative sampling* finds the optimal low-approximation of the PMI according to dependency measure between words and their context



Experimental evaluation ? More complicated.

Issues with instrisic evaluation

Problems With Evaluation of Word Embeddings Using Word Similarity (Faruqi et al, 2016)

- **Low correlation** with extrinsic tasks
- Absence of statistical significance, too many datasets
- *Frequency effects* in cosinus similarity
  - Words will similar frequencies will be closer in vector space
- Word similarity is **subjective** and hard to interpret:

$Word_1$	$Word_2$	Similarity score [0,10]
love	sex	6.77
stock	jaguar	0.92
money	cash	9.15
development	issue	3.97
lad	brother	4.46

Semantics derived automatically from language corpora contain human-like biases (Caliskan et al, 2017)

Bias in word embeddings: How to measure it ?

- **Word Embedding Association Test (WEAT)**

On Bias in word embeddings

WEAT:

- Sums the cosine distance between a set of words caracterising attributes (i.e, *gender*) and words to study
- Find axis of bias by reducing the dimension on a set of words differences ("*she*" - "*he*", "*her*" - "*him*", etc. . .)

Man is to Computer Programmer as Woman is to Homemaker ? Debiasing Word Embeddings (Bolukbasi et al, 2016)

Debiasing methods: usually based on removing (for example, through orthogonal projection) the information on that axis

Lipstick on a Pig: Debiasing Methods Cover up Systematic Gender Biases in Word Embeddings But do not Remove Them (Gonen and Goldberg, 2019)

$\rightarrow$ Debiasing this way does not work that well

Language (Technology) is Power: A Critical Survey of "Bias" in NLP (Blodgett et al, 2020)

$\rightarrow$ A set of recommendation on approaching bias in NLP

Conclusion and what's next

Sentence representations from words

1. We saw **symbolic representations**: BoW, TF-IDF...
   - Very large: need to limit the numbers of features
2. Then, **bottom-up representations from word embeddings**
   - Composition: use the mean of word representations.
   - It turns out simple baselines are already very good:
     - Use the **barycenter** with frequency weights
     - Use a **random projection** in a very large space

- **Principle of Compositionality**: the meaning of a complex expression is derivable from the meanings of its constituents and the rules used to combine them
  - Not so easy: *Figurative language*, *Implicitness*, Arising meaning
  - Next step: use neural architectures (CNN, RNN, Transformers) use to **compose representation**

Sequence modeling: what's next ?

We focused on **text representations** that could be **learnt directly**

- Symbolic document representations
- Generative models **assuming word independance**
- Dense, distributed word representations
  - Learnt from the distribution of the context (Word2vec, Glove)
  - By analogy, application of the same methods to *sentences*
- Lab: visualize several types of embeddings on several datasets
  - Look at *how well the features seem to align with the labels*
- Next step: how to model text sequences for complex tasks ?
  - Language modeling with deep learning architectures
  - Concepts of encoder and decoder models
  - Contextual language models